Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 43: *Mid-way evaluation and Machine Learning Intro*:
October 20, 2026

CC BY Licence 4.0: Lau Møller Andersen 2026

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects?

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous?

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data?

Week 47:

Lesson 9: Neural networks

Class 9: Code review

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything!

CC BY Licence 4.0: Lau Møller Andersen 2026

Recap

- Bias can be added in ways that improve prediction
  - it does this by removing collinearity
  - thereby making the model stable
  - and more generalisable

CC BY Licence 4.0: Lau Møller Andersen 2026

Learning goals and outline

Mid-way evaluation and Machine Learning Intro

1) Learning some early *classification* methods

— Perceptron and ADAline

— Classification depends on having a quantiser function

—Predicting categories instead of response

CC BY Licence 4.0: Lau Møller Andersen 2026

Mid-way evaluation

~10 min

- Write one thing you liked about the course so far
- What one thing would you change?
  - Be actionable!

CC BY Licence 4.0: Lau Møller Andersen 2026

I’ll summarise the feedback on the two points, and what we can change, next lecture

CC BY Licence 4.0: Lau Møller Andersen 2026

Encoding vs decoding

- ENCODING
  - Describes category
  - Rich data necessitates strategies for controlling the Family-wise error rate
  - Allows for inferences about specific features, e.g. *channel, voxel, frequency, time*
- DECODING
  - Predicts category
  - Rich data can be reduced to a few data points, reducing the impact of the Family-wise error rate
  - Does not (easily) allow for inferences about specific features (black box)

CC BY Licence 4.0: Lau Møller Andersen 2026

Let’s have a look at an iris flower

se·pal (sē′pəl)

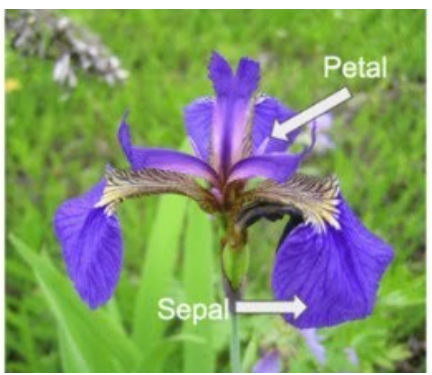
n.

One of the usually green leaflike structures composing the outermost part of a flower. Sepals often enclose and protect the bud and may remain after the fruit forms.

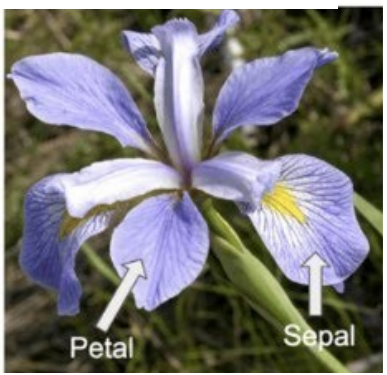
pet·al (pĕt′l)

n.

One of the often brightly colored parts of a flower immediately surrounding the reproductive organs; a division of the corolla.





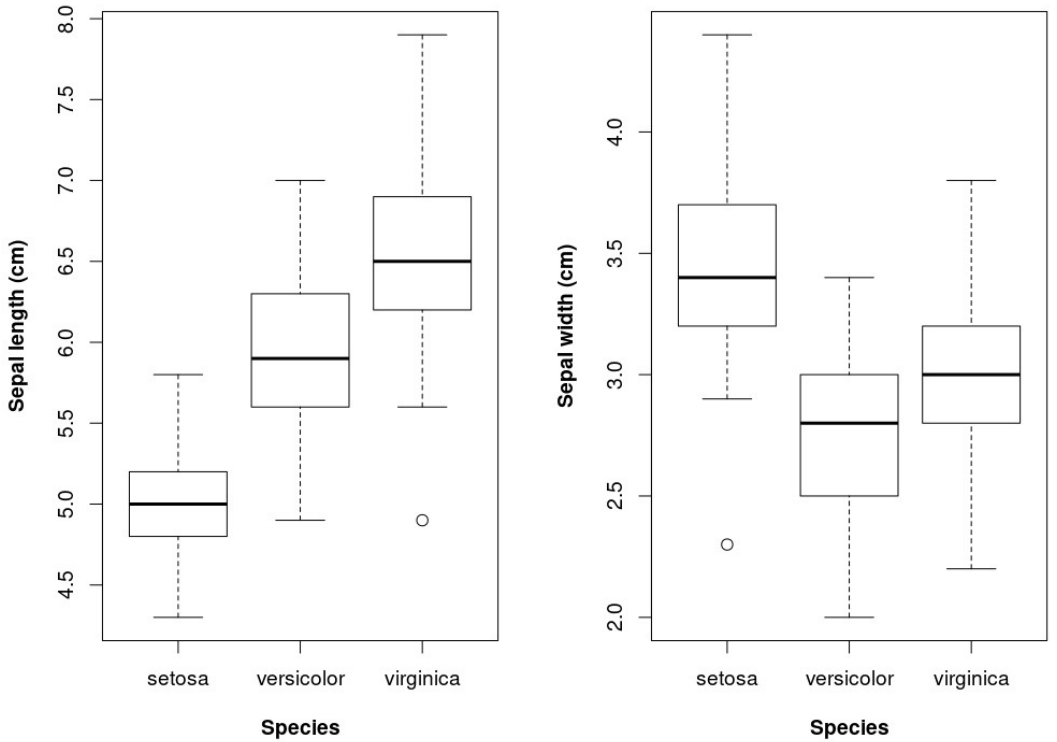


CC BY Licence 4.0: Lau Møller Andersen 2026

https://i.ytimg.com/vi/E8Nrl9QLNq8/maxresdefault.jpg

Encoding

Encoding: *describing* the *category* based on the *measurement* variable



CC BY Licence 4.0: Lau Møller Andersen 2026

Encoding

```
> print(anova(lm(Sepal.Length ~ Species, data=iris)))
Analysis of Variance Table

Response: Sepal.Length
           Df Sum Sq Mean Sq F value    Pr(>F)
Species     2 63.212  31.606  119.26 < 2.2e-16 ***
Residuals 147 38.956   0.265
```

```
> print(anova(lm(Sepal.Width ~ Species, data=iris)))
Analysis of Variance Table

Response: Sepal.Width
           Df Sum Sq Mean Sq F value    Pr(>F)
Species     2 11.345  5.6725   49.16 < 2.2e-16 ***
Residuals 147 16.962  0.1154
```

Encoding: *statistics* can then help you make *inferences* as to whether *measurements* differ between the *categories,* i.e. whether *categories* capture meaningful differences

CC BY Licence 4.0: Lau Møller Andersen 2026

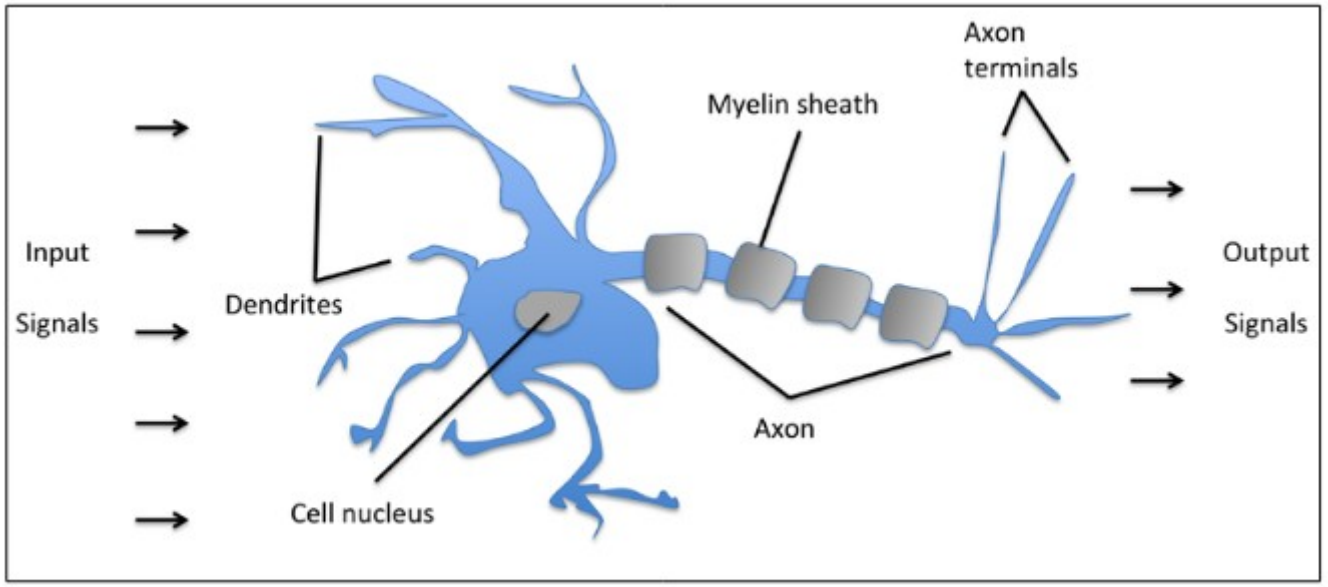
The Perceptron

DECODING

Raschka S (2015) Python Machine Learning. Packt Publishing Ltd

CC BY Licence 4.0: Lau Møller Andersen 2026

Black box idea



$$\boldsymbol{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

$$z = w_1 x_1 + \ldots + w_m x_m$$

Question: what do *x*, *w* and *z* correspond to in the above picture of the *Perceptron*?

(p. 18: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Prediction/classification rule

$$\phi(z) = \begin{cases} 1 \ \ if \ z \geq \theta \\ -1 \ \ otherwise \end{cases}$$

Perceptron fires

Perceptron doesn't fire

θ is a pre-specified threshold

(p. 18: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Prediction/classification rule

$$w_0 = -\theta$$

$$x_0 = 1$$

$$z = w_0 x_0 + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

$$z = -\theta + w_1 x_1 + \ldots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$$

$$\phi(z) = \begin{cases} 1 \;\; if \; z \geq 0 \\ -1 \;\; otherwise \end{cases}$$

Perceptron fires

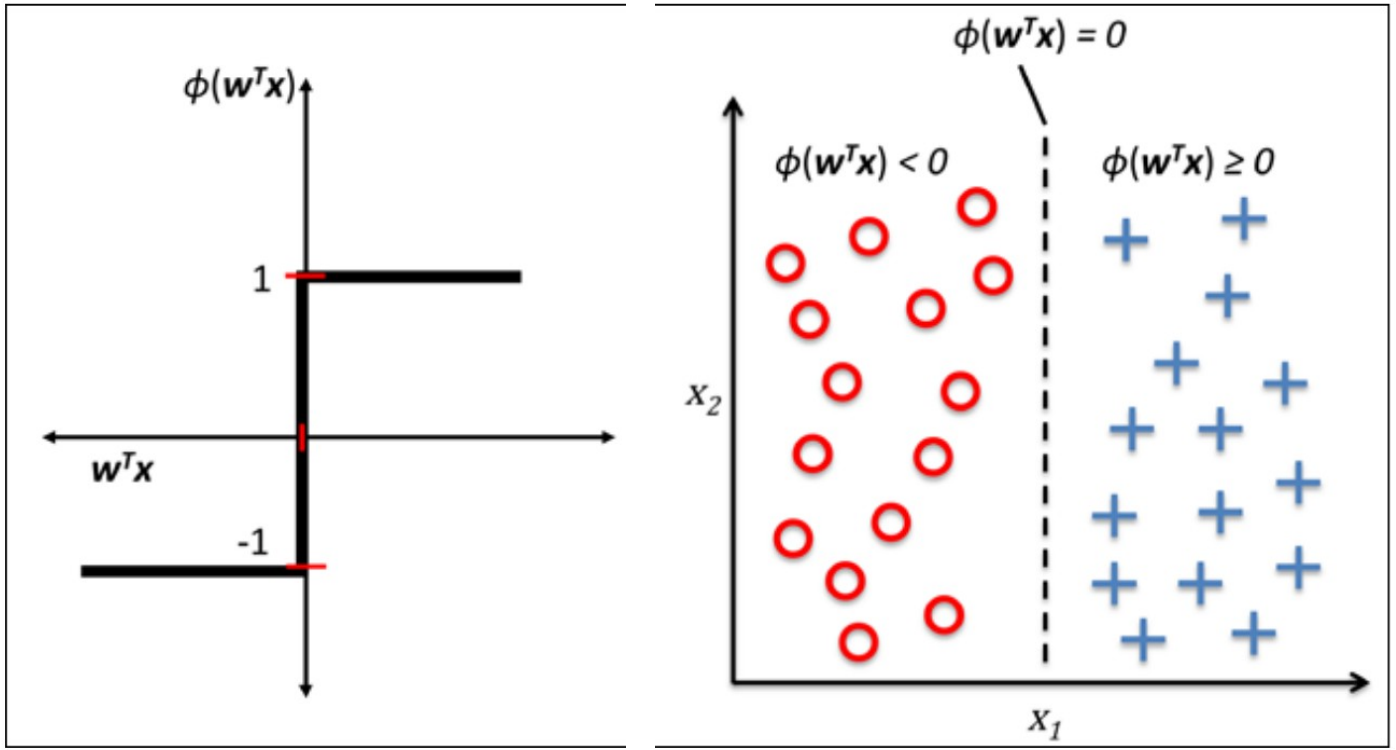
Perceptron doesn't fire

(p. 18: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Perceptron classification

We want to find $\boldsymbol{w^T x}$ that achieves this separation



(p. 21: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

In *Python* (in 2021)

```
class Perceptron(object):
    """ Perceptron classifier

    Parameters
    ------------
    eta : float
        Learning rate (between 0.0 and 1.0)
    n_iter : int
        Passes over the training dataset.

    Attributes
    -----------
    w_ : 1d-array
        Weights after fitting.
    errors_ : list
        Number of misclassifications in every epoch.

    """
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Special definition that indicates what the object (*Perceptron*) can be initialised with

```
def __init__(self, eta=0.01, n_iter=10):
    self.eta = eta
    self.n_iter = n_iter
```

```
ppn = Perceptron(eta=0.1, n_iter=10)
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Specifying methods of *Perceptron*

1. Initialize the weights to 0 or small random numbers.
2. For each training sample $\boldsymbol{x}^{(i)}$ perform the following steps:
    1. Compute the output value $\hat{y}$.
    2. Update the weights.

```python
def fit(self, X, y):
    """ Fit training data.

    Parameters
    ----------
    X : {array-like}, shape = [n_samples, n_features]
        Traing vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -------
    self : object

    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.errors_ = []

    for _ in range(self.n_iter):
        errors = 0
        for xi, target in zip(X, y):
            update = self.eta * (target - self.predict(xi))
            self.w_[1:] += update * xi
            self.w_[0] += update
            errors += int(update != 0.0)
        self.errors_.append(errors)
    return self
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Compute the output value $\hat{y}$

```python
def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.net_input(X) >= 0.0, 1, -1)
```

$$\phi(z) = \begin{cases} 1 & if\ z \geq \theta \\ -1 & otherwise \end{cases}$$

CC BY Licence 4.0: Lau Møller Andersen 2026

When we are right

real label

$$\Delta w_j = \eta\left(-1 - (-1)\right) x_j^{(i)} = 0$$

predicted label

```
update = self.eta * (target - self.predict(xi))
self.w_[1:] += update * xi
self.w_[0] += update
```

real label

$$\Delta w_j = \eta\left(1 - 1\right) x_j^{(i)} = 0$$

predicted label

Δw_j: change in weight

η: learning rate

CC BY Licence 4.0: Lau Møller Andersen 2026

When we are wrong

real label

$$\Delta w_j = \eta\left(1 - (-1)\right) x_j^{(i)} = \eta(2)\, x_j^{(i)}$$

predicted label

```
update = self.eta * (target - self.predict(xi))
self.w_[1:] += update * xi
self.w_[0] += update
```

real label

$$\Delta w_j = \eta\left(-1 - 1\right) x_j^{(i)} = \eta(-2)\, x_j^{(i)}$$

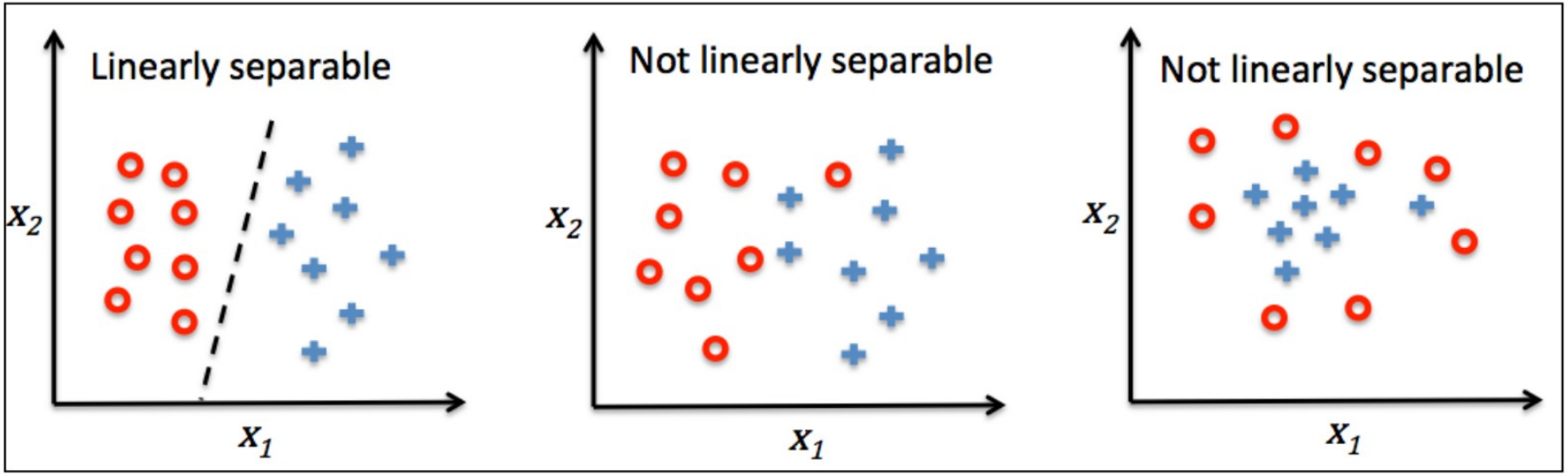
predicted label

Δw_j: change in weight

η: learning rate

CC BY Licence 4.0: Lau Møller Andersen 2026

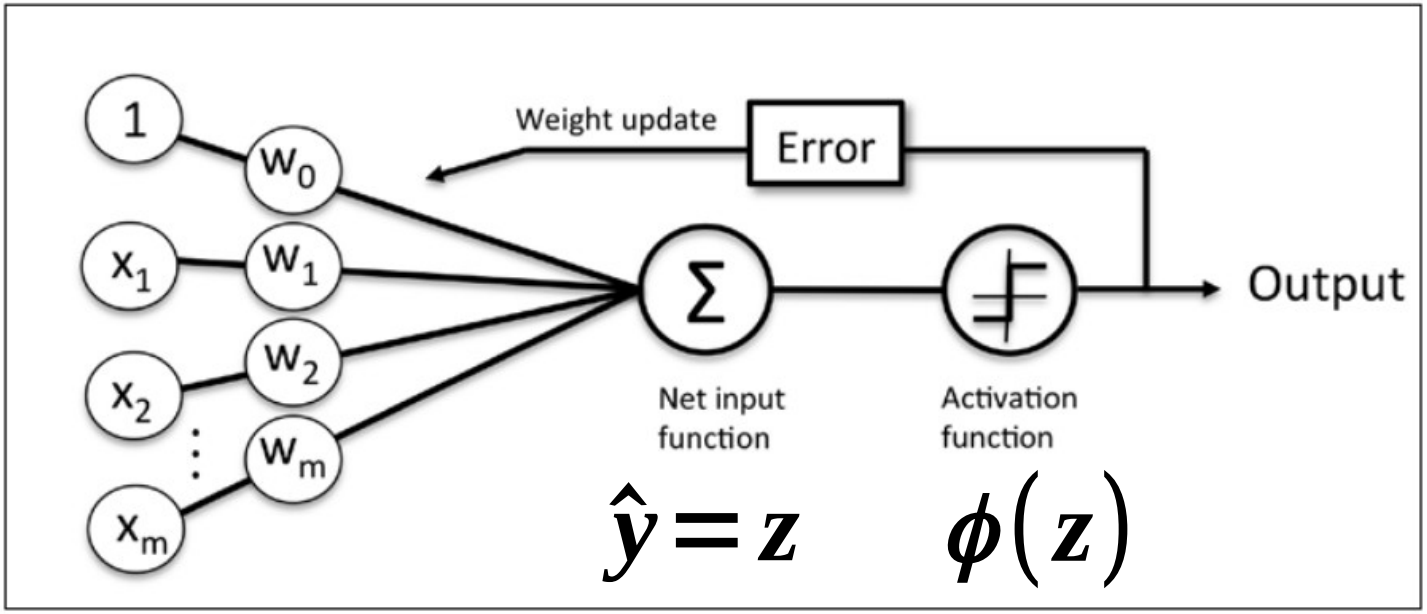
Convergence only possible when linearly separable



(p. 23: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Perceptron: Graphical summary

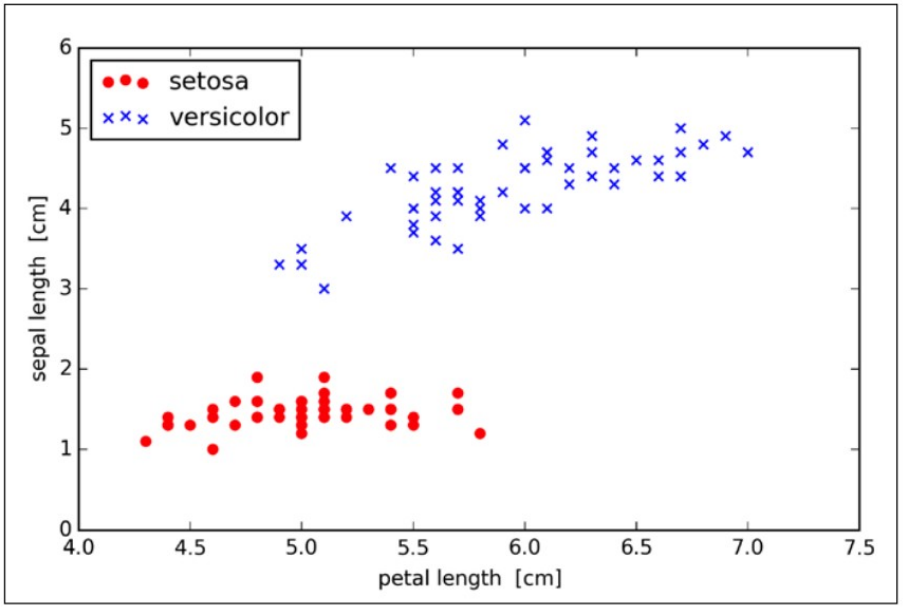


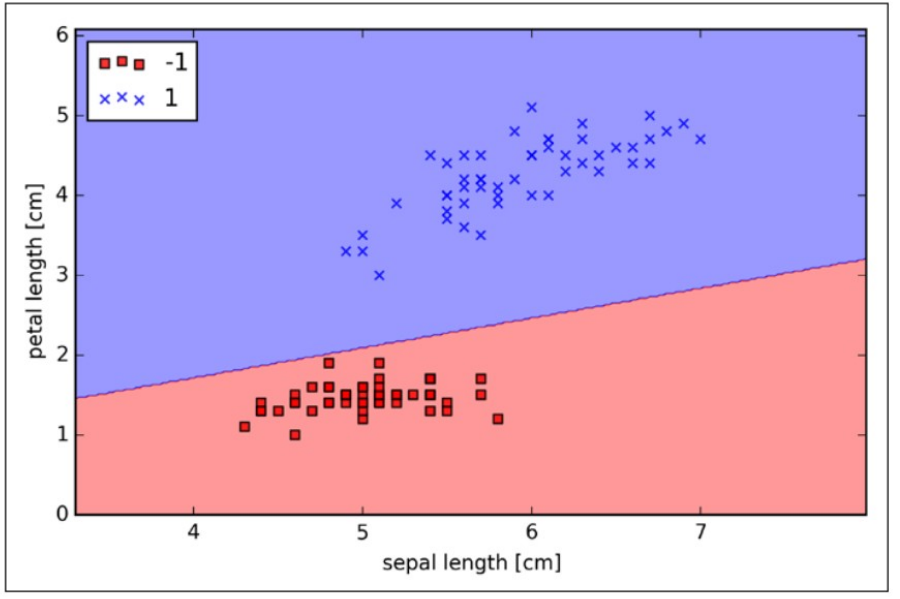
(p. 24: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

An example

CC BY Licence 4.0: Lau Møller Andersen 2026






```
In [153]: ppn.w_
Out[153]: array([-0.04 , -0.068,  0.182])
```

```
In [155]: X_subset[0, :]
Out[155]: array([5.1, 1.4])
```

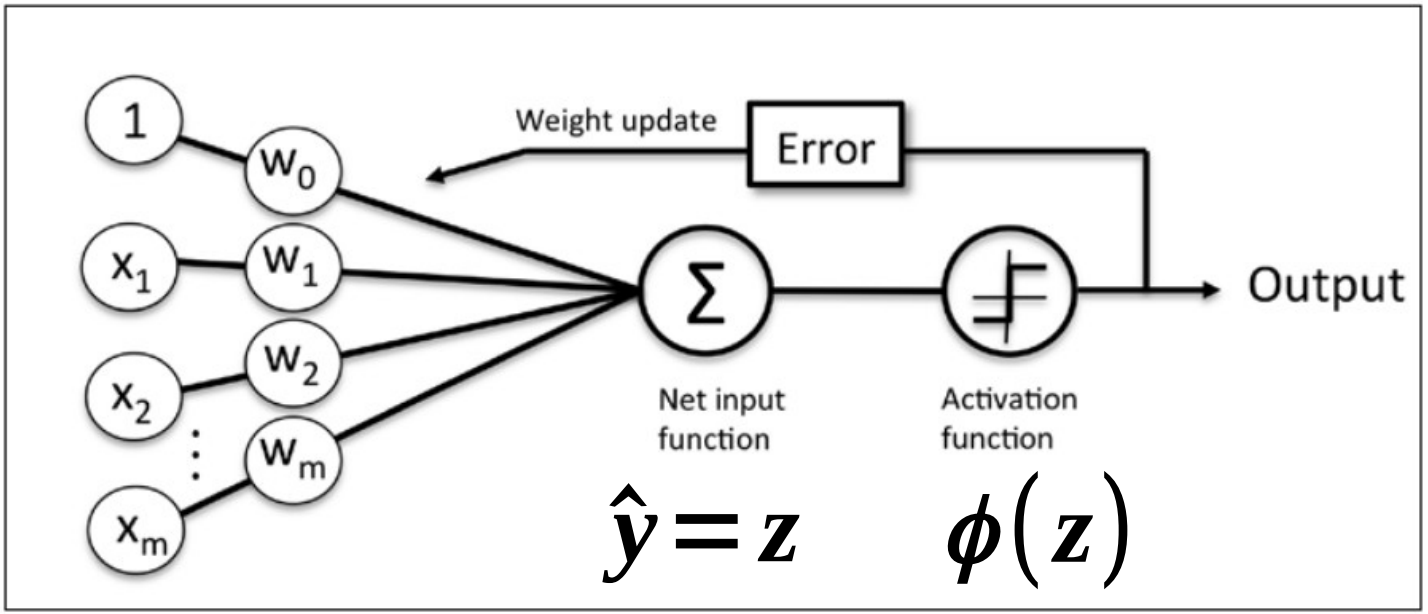
$$\hat{y}_1 = -0.04 - 0.068 \cdot 5.1 + 0.182 \cdot 1.4 = -0.132$$

```
In [156]: ppn.net_input(X_subset[0, :])
Out[156]: -0.13199999999999953
```

(p. 29 & p. 32: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Perceptron: Graphical summary



(p. 24: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Live coding Perceptron
(*W43_live_coding.py*)

CC BY Licence 4.0: Lau Møller Andersen 2026

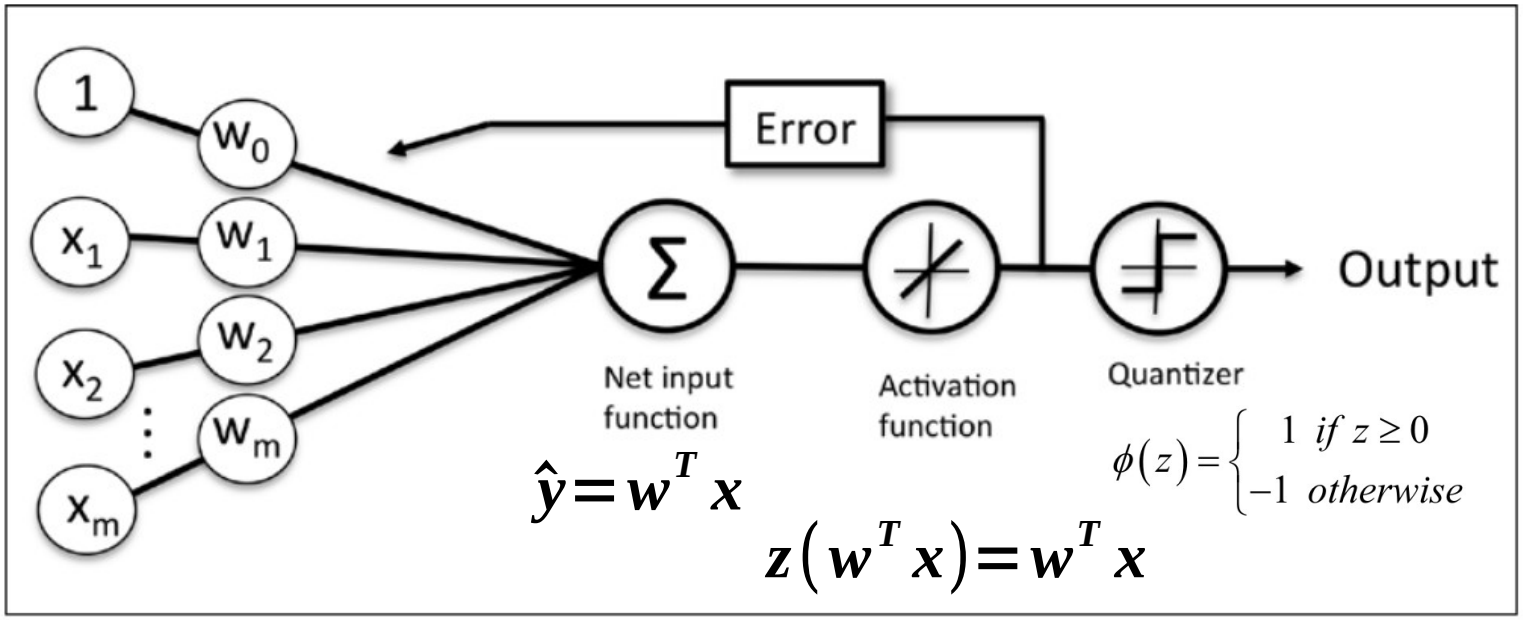
ADALINE **ADA**ptive **LI**near **NE**uron

DECODING

Raschka S (2015) Python Machine Learning. Packt Publishing Ltd

CC BY Licence 4.0: Lau Møller Andersen 2026

ADAptive LInear NEuron (ADALINE)



$$w^T x = w_0 x_0 + w_1 x_1 + \ldots + w_{m-1} x_{m-1} + w_m x_m$$

CC BY Licence 4.0: Lau Møller Andersen 2026

(p. 33: Raschka, 2015)

ADALINE Gradient descent

```
def __init__(self, eta=0.01, n_iter=50):
    self.eta = eta
    self.n_iter = n_iter
```

```
class AdalineGD(object):
    """ ADAptive LInear NEuron classifier

    Parameters
    ------------
    eta : float
        Learning rate (between 0.0 and 1.0)
    n_iter : int
        Passes over the training dataset.

    Attributes
    -----------
    w_ : 1d-array
        Weights after fitting.
    errors_ : list
        Number of misclassifications in every epoch.

    """
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Methods

$$\hat{y} = w^T x$$

$$z\left(w^T x\right) = w^T x$$

$$\phi(z) = \begin{cases} 1 \ \ if \ z \geq 0 \\ -1 \ \ otherwise \end{cases}$$

```python
def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def activation(self, X):
    """Computer linear activation"""
    return self.net_input(X)

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

CC BY Licence 4.0: Lau Møller Andersen 2026

The *fit* method

eta (η): learning rate (a constant)

output: $X\hat{\beta} = \hat{y}$

errors: $y - \hat{y}$

X.T.dot(errors): $X^T \cdot (y - \hat{y})$

$X^T \cdot (y - \hat{y}) = \Delta w_1 + \Delta w_2 + ... + \Delta w_{m-1} + \Delta w_m$

cost function: $(\sum (y - \hat{y})^2)/2$

```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    ----------
    X : {array-like}, shape = [n_samples, n_features]
        Traing vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -------
    self : object

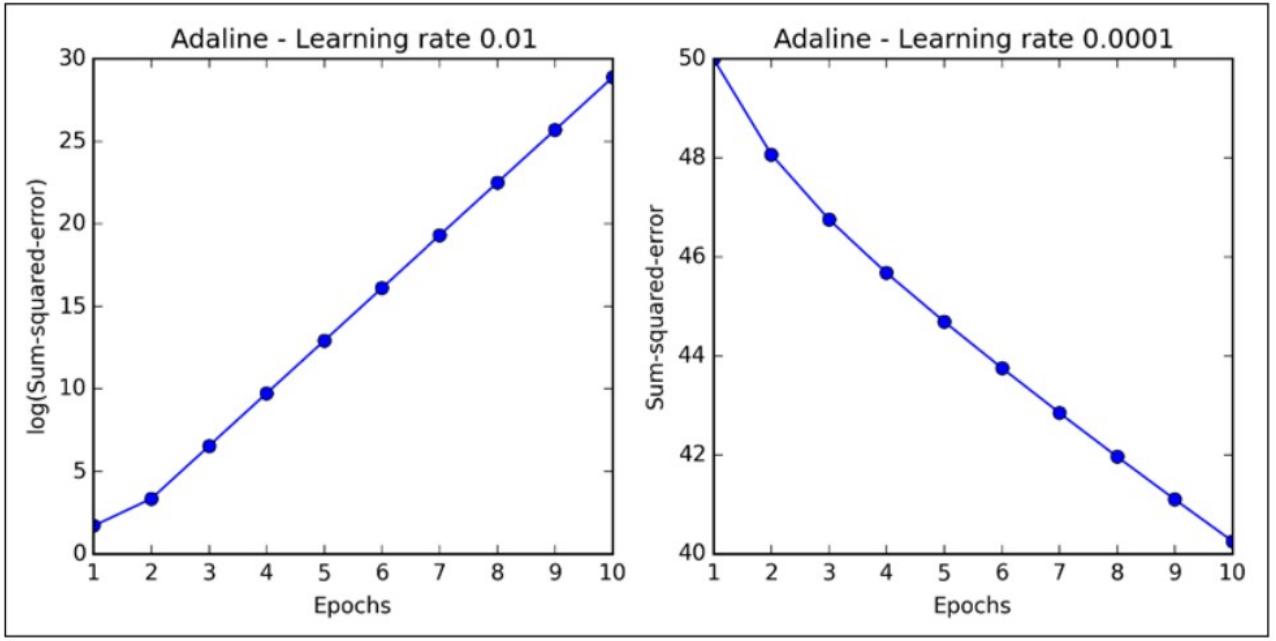
    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Learning rate

Overshooting the global minimum



Slow stepping down

Always check whether it converges

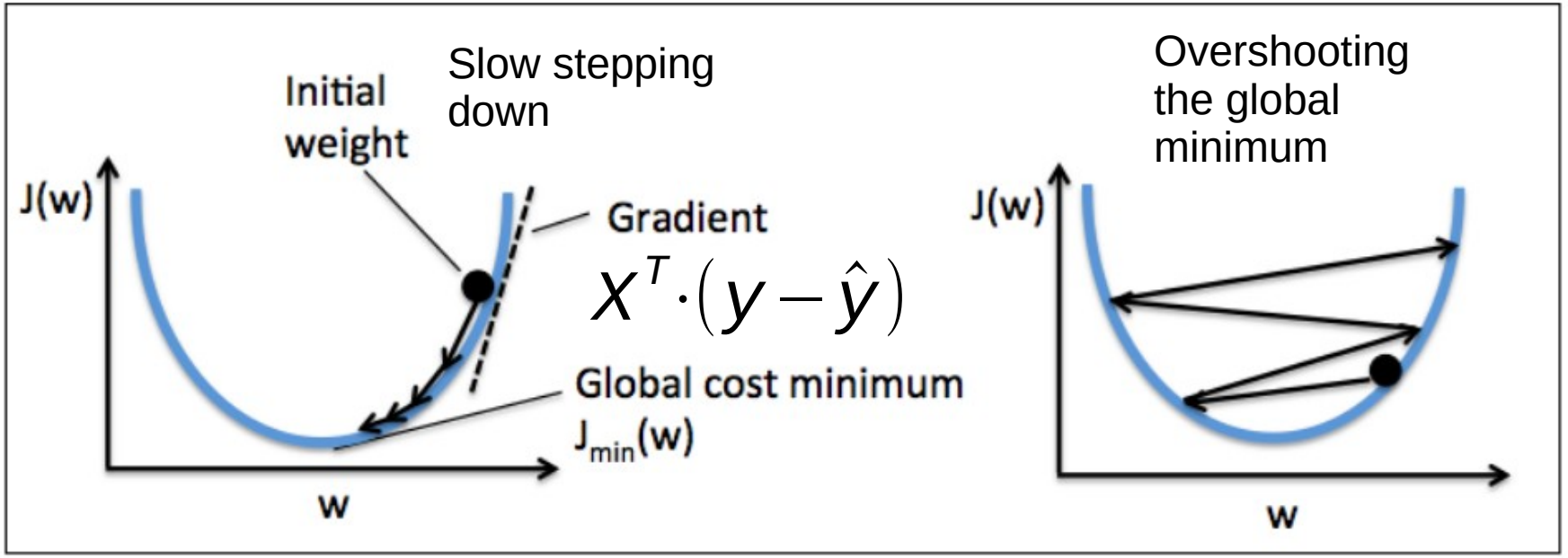
(p. 40: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026

Gradient descent,

```
self.w_[1:] += self.eta * X.T.dot(errors)
self.w_[0] += self.eta * errors.sum()
```

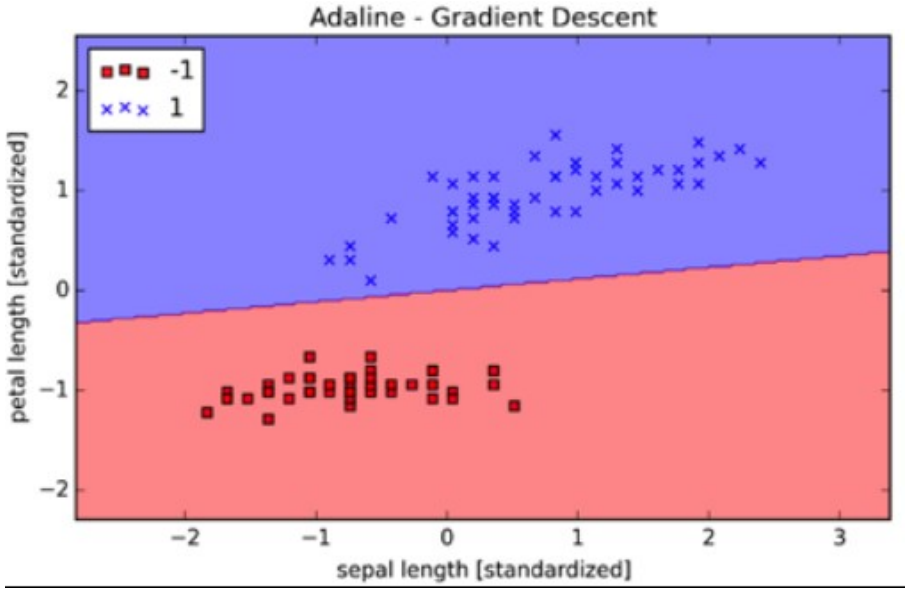
SETTING η (ETA) OPTIMALLY



$$J(w) = \left(\sum (y - \hat{y})^2\right)/2$$

(p. 40: Raschka, 2015)

CC BY Licence 4.0: Lau Møller Andersen 2026



(p. 42: Raschka, 2015)

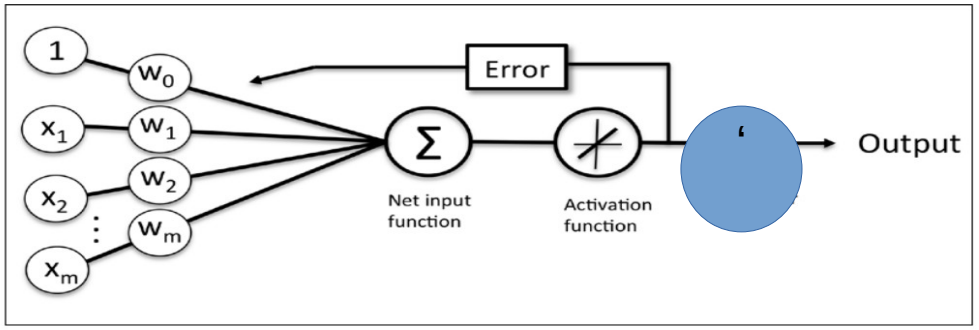
```
In [163]: ada.w_
Out[163]: array([-0.40808285, -0.33924452,  0.79202224])
```

```
In [155]: X_subset[0, :]
Out[155]: array([5.1, 1.4])
```

$$\hat{y}_1 \approx -0.408 - 0.339 \cdot 5.1 + 0.792 \cdot 1.4 = -1.03$$

```
In [164]: ada.net_input(X_subset[0, :])
Out[164]: -1.029398778794785
```

CC BY Licence 4.0: Lau Møller Andersen 2026



How does this relate to linear regression?

From ADALINE

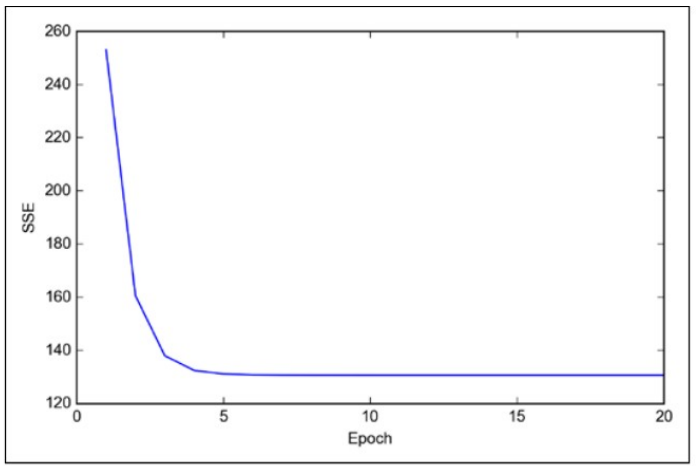
$$\boldsymbol{w}^{\boldsymbol{T}}\ \boldsymbol{x} = w_0\ x_0 + w_1\ x_1 + \ldots + w_{m-1}\ x_{m-1} + w_m\ x_m$$

CC BY Licence 4.0: Lau Møller Andersen 2026

Very similar to ADALINE
and when converged will be virtually identical to the ordinary least squares solution

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Convergence




```python
class LinearRegressionGD(object):

    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        self.w_ = np.zeros(1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            output = self.net_input(X)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def predict(self, X):
        return self.net_input(X)
```

CC BY Licence 4.0: Lau Møller Andersen 2026

ADALINE

```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    ----------
    X : {array-like}, shape = [n_samples, n_features]
        Traing vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -------
    self : object

    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

```
class LinearRegressionGD(object):

    def __init__(self, eta=0.001, n_iter=20):
        self.eta = eta
        self.n_iter = n_iter

    def fit(self, X, y):
        self.w_ = np.zeros(1 + X.shape[1])
        self.cost_ = []

        for i in range(self.n_iter):
            output = self.net_input(X)
            errors = (y - output)
            self.w_[1:] += self.eta * X.T.dot(errors)
            self.w_[0] += self.eta * errors.sum()
            cost = (errors**2).sum() / 2.0
            self.cost_.append(cost)
        return self

    def net_input(self, X):
        return np.dot(X, self.w_[1:]) + self.w_[0]

    def predict(self, X):
        return self.net_input(X)
```

```
def net_input(self, X):
    """Calculate net input"""
    return np.dot(X, self.w_[1:]) + self.w_[0]

def activation(self, X):
    """Computer linear activation"""
    return self.net_input(X)

def predict(self, X):
    """Return class label after unit step"""
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

CC BY Licence 4.0: Lau Møller Andersen 2026

Live coding ADALINE

(*W43_live_coding.py*)

CC BY Licence 4.0: Lau Møller Andersen 2026

Interim summary

- We can build simple classification tools using scikit-learn
  - These can give us decision boundaries
  - That we can apply to new data (we haven’t done that yet)
- We need to define cost functions
  - These can be conceptually separated from prediction functions (remember ADALINE)

CC BY Licence 4.0: Lau Møller Andersen 2026

Summary

- We can build simple classification tools using scikit-learn
  - These can give us decision boundaries
  - That we can apply to new data (we haven’t done this yet)
- We need to define cost functions
  - These can be conceptually separated from prediction functions (remember ADALINE)

CC BY Licence 4.0: Lau Møller Andersen 2026

Learning goals and outline

Mid-way evaluation and Machine Learning Intro

1) Learning some early *classification* methods

— Perceptron and ADAline

— Classification depends on having a quantiser function

—Predicting categories instead of response

CC BY Licence 4.0: Lau Møller Andersen 2026

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects?

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous?

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data?

Week 47:

Lesson 9: Neural networks

Class 9: Code review

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything!

CC BY Licence 4.0: Lau Møller Andersen 2026

Next time

- Logistic regression
  - adapted for machine learning
  - aim: predicting categories instead of minimising log-likelihood when generalised to new datasets
- Linear regression
  - adapted for machine learning
  - aim: minimising error when generalised to new datasets

CC BY Licence 4.0: Lau Møller Andersen 2026

Reading questions

- Chapter 10
  - What is lasso and ridge regression?
  - What is standardisation?
- Chapter 3
  - What is the cost function of logistic regression?
  - What is overfitting and underfitting?

CC BY Licence 4.0: Lau Møller Andersen 2026